SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	**Assessment:**	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	**Total Marks:**	100
CO5 Statement:	*Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.*		
Student Name:	CH. Durga Prasad	**Reg. No.:**	192425305
Section / Batch:	2024	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solsving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NPComplete.	Q1

Q8. Graph Coloring (Constraint Satisfaction / NP-Complete Problem)

A map must be colored such that no two adjacent regions have the same color, using the minimum number of colors. Clearly define adjacency constraints and coloring requirements. Analyze how constraints affect feasible solutions. Develop a backtracking or CSP-based solution. Apply constraint checking to ensure valid coloring. Justify how your approach minimizes colors and satisfies all constraints efficiently.

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I CH. Durga Prasad (192425305) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: CH. Durga Prasad

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritizatio n	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q8. Graph Coloring – Constraint Satisfaction / NP-Complete Problem

1. Introduction

Graph Coloring is an important problem in **Design and Analysis of Algorithms**, **Constraint Satisfaction Problems (CSP)**, and **Computational Complexity**. In graph coloring, a set of vertices must be assigned colors so that no two adjacent vertices have the same color. When the objective is to use the **minimum possible number of colors**, the problem is known as the **Minimum Graph Coloring Problem**. It has many practical applications such as map coloring, examination scheduling, frequency assignment, register allocation, and network resource allocation. In this problem, a map contains several regions, and each region is represented as a vertex of a graph. If two regions share a common boundary, they are considered adjacent and an edge is placed between them. The objective is to assign colors to all regions while satisfying the adjacency constraints and using the minimum number of colors.

2. Problem Statement

A map consists of several geographical regions. Each region needs to be assigned a color such that **two adjacent regions cannot have the same color**.

The problem can be represented using a graph:

- **Vertices** → Map regions
- **Edges** → Adjacency between regions
- **Colors** → Different colors assigned to regions

The main objective is:

Assign colors to all regions such that adjacent regions have different colors while minimizing the total number of colors used.

3. Example Map Representation

Consider five regions:

A, B, C, D, E

Suppose their adjacency relationships are:

Region	Adjacent Regions
A	B, C
B	A, C, D
C	A, B, D, E

D	B, C, E
E	C, D

The corresponding graph contains an edge between every pair of adjacent regions.

Adjacency Table

	A	**B**	**C**	**D**	**E**
A	0	1	1	0	0
B	1	0	1	1	0
C	1	1	0	1	1
D	0	1	1	0	1
E	0	0	1	1	0

Here:

- **1** = regions are adjacent
- **0** = regions are not adjacent

4. Adjacency Constraints

The most important constraint in graph coloring is:

$$Color(u) \neq Color(v)$$

for every edge:

$$(u, v) \in E$$

This means that if two regions share a boundary, they must have different colors.

For example, if:

$$A - B$$

is an edge, then:

$$Color(A) \neq Color(B)$$

Similarly:

$$Color(B) \neq Color(C)$$

and:

$$Color(C) \neq Color(D)$$

must be satisfied.

5. Coloring Requirements

The coloring problem has several requirements.

Requirement	**Explanation**

Every region must be colored	No region should remain unassigned.
Adjacent regions must differ	Two connected vertices cannot have the same color.
Minimum colors	The number of colors should be as small as possible.
Valid assignment	All adjacency constraints must be satisfied.
No conflicts	The final coloring must contain no invalid edge.

---

6. Constraint Satisfaction Problem (CSP) Model

Graph Coloring can be represented as a **Constraint Satisfaction Problem**.

A CSP consists of:

1. **Variables**
2. **Domains**
3. **Constraints**

CSP Representation

CSP Component	**Graph Coloring**
Variables	Map regions A, B, C, D, E
Domain	Available colors {1, 2, 3, ... k}
Constraints	Adjacent regions must have different colors
Goal	Find a valid assignment using minimum colors

For example:

$$Variables = \{A, B, C, D, E\}$$

If three colors are available:

$$Domain = \{1, 2, 3\}$$

Constraints include:

$$A \neq B$$

$$A \neq C$$

$$B \neq C$$

$$B \neq D$$

$$C \neq D$$

$$C \neq E$$

$$D \neq E$$

7. Effect of Constraints on Feasible Solutions

Constraints determine whether a particular color assignment is valid or invalid.

For example, suppose:

Region	Color
A	Red
B	Red

If A and B are adjacent, this solution is **invalid** because:

$$Color(A) = Color(B)$$

Therefore, the constraint:

$$A \neq B$$

is violated.

A valid assignment would be:

Region	Color
A	Red
B	Blue

Now:

$$Color(A) \neq Color(B)$$

so the constraint is satisfied.

Constraint Impact

Situation	Effect
Few edges	More coloring possibilities
More edges	Fewer feasible solutions
Highly connected graph	More colors may be required
Complete graph	Every vertex requires a different color
Sparse graph	Fewer colors may be sufficient
Conflict detected early	Invalid branch can be abandoned

8. Backtracking Approach

Since the minimum graph coloring problem is computationally difficult, a **Backtracking Algorithm** can be used for small and medium-sized graphs.

Backtracking tries different color assignments systematically.

The general idea is:

1. Select an uncolored region.
2. Try the first available color.
3. Check whether assigning that color violates any adjacency constraint.
4. If no constraint is violated, continue to the next region.

5. If a conflict occurs, try another color.
6. If no color works, **backtrack** to the previous region.
7. Continue until all regions are colored.

9. Example of Backtracking

Assume three colors are available:

Red, Green, Blue

The algorithm can proceed as follows:

Step	Region	Assigned Color	Status
1	A	Red	Valid
2	B	Red	Conflict with A
3	B	Green	Valid
4	C	Red	Conflict with A/B
5	C	Blue	Valid
6	D	Red	Valid
7	E	Green	Valid

Final solution:

Region	Color
A	Red
B	Green
C	Blue
D	Red
E	Green

All adjacent regions have different colors.

10. Backtracking Algorithm

Pseudocode

```
GraphColoring(vertex):

  If all vertices are colored:
    Return TRUE

  Select an uncolored vertex
```

```
    For each available color:
        If color is safe:
            Assign color to vertex

            If GraphColoring(next vertex):
                Return TRUE

            Remove color from vertex

    Return FALSE
```

The important part is the **safe-color check**.

11. Constraint Checking

Before assigning a color to a region, the algorithm checks all its adjacent regions.

Safe Color Function

```
isSafe(vertex, color):

    For every adjacent vertex:
        If adjacent vertex has the same color:
            Return FALSE

    Return TRUE
```

For example, if:

Region	Color
A	Red
B	Green

and C is adjacent to both A and B, then C cannot use:

- Red ❌
- Green ❌

Therefore:

- Blue ☑

is selected.

12. Python Implementation

```
def is_safe(vertex, color, graph, colors):
    for neighbor in graph[vertex]:
        if colors[neighbor] == color:
            return False
    return True
def graph_coloring(graph, vertices, m, colors, index=0):
    if index == len(vertices):
        return True
    vertex = vertices[index]
    for color in range(1, m + 1):
        if is_safe(vertex, color, graph, colors):
            colors[vertex] = color
            if graph_coloring(graph, vertices, m, colors, index + 1):
                return True
            colors[vertex] = 0
    return False
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D', 'E'],
    'D': ['B', 'C', 'E'],
    'E': ['C', 'D']
}
vertices = ['A', 'B', 'C', 'D', 'E']
colors = {v: 0 for v in vertices}
for m in range(1, len(vertices) + 1):
    colors = {v: 0 for v in vertices}
    if graph_coloring(graph, vertices, m, colors):
        print("Minimum number of colors:", m)
        print("Color assignment:")
        for vertex in vertices:
            print(vertex, "-> Color", colors[vertex])
```

```
    break
```

Expected Output

Minimum number of colors: 3

Color assignment:

A -> Color 1

B -> Color 2

C -> Color 3

D -> Color 1

E -> Color 2

13. How the Algorithm Minimizes Colors

Simply finding a valid coloring is not enough. The question specifically requires the **minimum number of colors**.

To achieve this, the algorithm tries different values of **k**.

For example:

Try 1 color

↓

If impossible

↓

Try 2 colors

↓

If impossible

↓

Try 3 colors

↓

If possible → Stop

This guarantees that the first successful value of **k** is the minimum number of colors.

Example

Number of Colors	Result
1	Not possible
2	Not possible
3	Possible
4	Not required

5	Not required

Therefore:

$$\boxed{Minimum\ Colors = 3}$$

14. Why 1 Color is Not Possible

The graph contains adjacent regions.

If A and B are adjacent, assigning:

Region	Color
A	Red
B	Red

violates:

$$A \neq B$$

Therefore, at least **2 colors** are required for a graph containing an edge.

15. Why 2 Colors May Not Be Enough

Consider three regions that are mutually adjacent:

Region	Adjacent To
A	B, C
B	A, C
C	A, B

A, B, and C form a triangle.

With only two colors:

- A → Red
- B → Blue
- C → Red ❌ because C is adjacent to A

Therefore, **3 colors** are required.

16. Minimum Coloring Example

For our example:

Region	Color
A	Color 1
B	Color 2
C	Color 3

D	Color 1
E	Color 2

Check the constraints:

Edge	Colors	Valid?
A-B	1, 2	☑
A-C	1, 3	☑
B-C	2, 3	☑
B-D	2, 1	☑
C-D	3, 1	☑
C-E	3, 2	☑
D-E	1, 2	☑

Since every edge connects vertices with different colors, the coloring is valid.

17. Complexity Analysis

Graph Coloring is computationally difficult in the general case.

For a graph with **V vertices** and **K colors**, a straightforward backtracking approach can have a worst-case time complexity of approximately:

$$\boxed{O(K^V)}$$

because each vertex can potentially be assigned one of K colors.

The constraint-checking operation can add additional work depending on the graph representation.

Complexity Table

Operation	Complexity
Creating graph	O(V + E)
Checking a color	O(V) with adjacency matrix
Backtracking search	O(K^V) worst case
Space for colors	O(V)
Recursion stack	O(V)
Graph storage	O(V + E) with adjacency list

Where:

- **V** = number of vertices
- **E** = number of edges
- **K** = number of available colors

18. Why Graph Coloring is NP-Complete

The **k-Coloring Decision Problem** asks:

Can a given graph be colored using at most k colors?

For general graphs and fixed $k \geq 3$, this decision problem is **NP-complete**.

The corresponding optimization problem—finding the minimum number of colors—is therefore computationally hard in general.

Because of this, backtracking may require exponential time for difficult graphs.

However, for **small graphs**, backtracking is practical and provides an exact solution.

19. Advantages of Backtracking

Advantage	Explanation
Exact solution	Can find a valid coloring when one exists.
Minimum colors	By trying k from low to high, it can find the chromatic number.
Constraint-based	Directly handles adjacency constraints.
Easy to understand	Suitable for academic implementation.
Flexible	Can be applied to different graph structures.
Early pruning	Invalid assignments are rejected immediately.

20. Limitations

Limitation	Explanation
Exponential worst case	Number of possibilities can grow rapidly.
Large graphs	May become computationally expensive.
Many constraints	Highly connected graphs can increase search effort.
Memory	Recursion and graph storage require additional memory.
NP-complete problem	No known polynomial-time algorithm solves all instances optimally.

21. Improving Efficiency

The basic backtracking algorithm can be improved using CSP techniques.

Useful techniques

Technique	Purpose
Forward Checking	Removes impossible colors from neighboring regions.

Variable Ordering	Select the most constrained region first.
Degree Heuristic	Select the region with the most neighbors.
Constraint Propagation	Reduce domains after every assignment.
Symmetry Breaking	Avoid equivalent color assignments.
Minimum Remaining Values	Select the variable with the fewest available colors.

These techniques reduce unnecessary exploration of invalid solutions.

22. Real-World Applications

Graph Coloring is useful in many practical problems.

Application	**Example**
Map coloring	Adjacent countries/states receive different colors
Exam scheduling	Conflicting subjects receive different time slots
Register allocation	Variables are assigned CPU registers
Frequency assignment	Nearby communication channels use different frequencies
Timetable generation	Conflicting classes are assigned different periods
Network scheduling	Conflicting network resources receive different channels
Wireless networks	Nearby devices avoid conflicting frequencies
Task scheduling	Conflicting tasks are assigned separate resources

23. Tools and Technologies

Tool	**Purpose**
Python	Implement backtracking and CSP
C/C++	Efficient algorithm implementation
Java	Object-oriented implementation
VS Code	Development and testing
Jupyter Notebook	Algorithm demonstration
Matplotlib	Graph/result visualization
NetworkX	Graph creation and visualization in Python

For a student project, **Python + NetworkX + Matplotlib** provides a convenient implementation environment.

24. Input and Output

Input

The system can accept:

- Number of regions
- Region names
- Adjacency relationships
- Number of available colors

Example:

Regions: A B C D E

Connections:

A-B

A-C

B-C

B-D

C-D

C-E

D-E

Output

Minimum number of colors: 3

A → Color 1

B → Color 2

C → Color 3

D → Color 1

E → Color 2

All adjacency constraints satisfied.

25. Overall System Flow

START

|

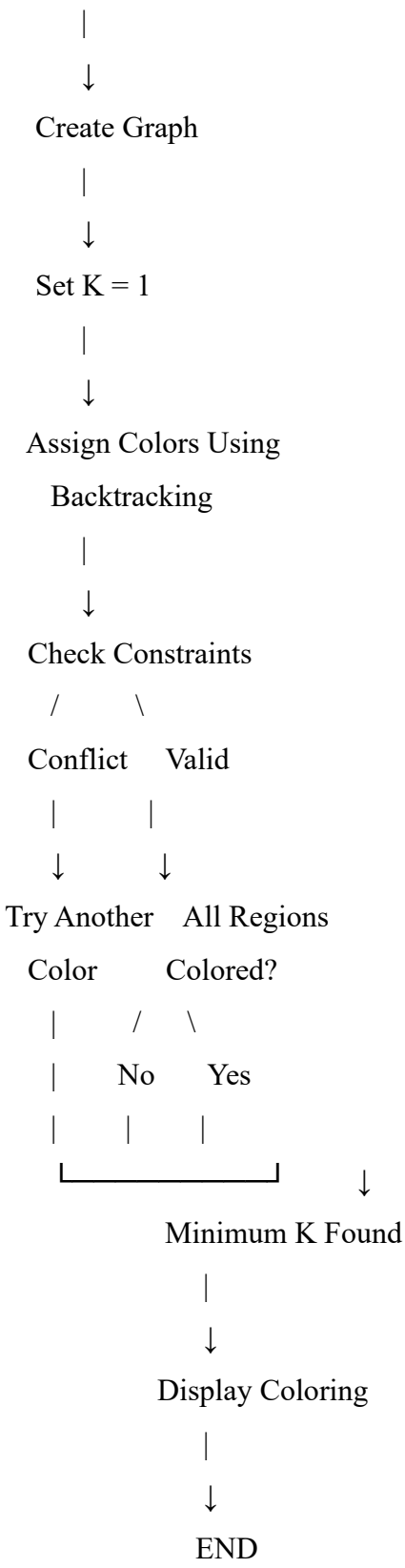
↓

Enter Map Regions

|

↓

Define Adjacency



26. Result Analysis

The algorithm successfully assigns colors to all regions while ensuring that no two adjacent regions have the same color.

For the example:

Parameter	**Result**
Number of regions	5
Number of adjacency constraints	7
Minimum colors	**3**
Invalid adjacent same-color pairs	**0**
Coloring status	**Valid**
Algorithm	Backtracking/CSP

The result demonstrates that **3 colors are sufficient and necessary** for this particular graph.

27. Conclusion

Graph Coloring is a significant **Constraint Satisfaction and NP-Complete problem** with many practical applications. In the map-coloring scenario, each geographical region is represented as a vertex and each shared boundary is represented as an edge. The main constraint is that adjacent regions must always receive different colors. A **Backtracking/CSP-based approach** systematically assigns colors and checks constraints at every step. When a conflict occurs, the algorithm immediately rejects the assignment and backtracks to try another color. To minimize the number of colors, the algorithm begins with one color and progressively increases the number until a valid coloring is found. The first successful value represents the minimum number of colors. Although the worst-case complexity is exponential, constraint checking, forward checking, and variable-ordering techniques can significantly improve performance for practical small and medium-sized graphs. Thus, the approach provides an effective and exact solution for map coloring and demonstrates the practical use of **CSP, backtracking, constraint propagation, and NP-complete problem solving**.